

Welcome to Weather App!

Choose a city from the list below to check the weather.

Seoul

Tokyo

Paris

London



Project Implementation Process

과제 구현 과정

Jieun Baek 백지은

github : https://github.com/Jieun13/Bzznbyd_assignment

1. 요구사항 정의

1. 프로젝트 주제

: OpenWeather API를 활용해서
4개 도시의 현재 날씨와
5일 간의 예보를 보여주는 웹 페이지 구현

2. 기술 스택

- 프레임워크 : Next.js 12 (Create Next App)
- FE : Apollo Client
 - 스타일 : Module CSS
- BE : GraphQL + Apollo Server
- 외부 API : OpenWeather
- 테스트 : Jest (선택) → 실제 미구현

3. 개발 규칙

1. 모듈 단위 개발
2. ES6 문법 사용
3. 시멘틱 태그 사용
4. 반응형 구현

3-1. 반응형 브레이크 포인트

1280px 이상	레이아웃 화면 가운데 정렬
800px 이상	레이아웃 width 100%
800px 미만	레이아웃 고정 크기, 스크롤 처리

1. 요구사항 정의

4. 기능적 요구사항

- 사용자는 localhost:3000으로 접속했을 때 메인 페이지를 확인할 수 있어야 한다.
- 메인 페이지에는 도시 목록 버튼(Seoul, Tokyo, Paris, London)이 표시되어야 한다.
- 사용자가 도시 버튼을 클릭하면 해당 도시의 상세 페이지로 이동해야 한다.
- 각 도시 상세 페이지는 /Seoul, /Tokyo, /Paris, /London 형식의 URL 구조로 구성되어야 한다.
- 상세 페이지에서는 선택한 도시의 현재 날씨 정보를 확인할 수 있어야 한다.
- 상세 페이지에서는 선택한 도시의 5일 예보 정보를 확인할 수 있어야 한다.
- 현재 날씨 정보는 OpenWeather의 Current Weather API를 사용하여 조회해야 한다.
- 5일 예보 정보는 OpenWeather의 3-hour Forecast 5 days API를 사용하여 조회해야 한다.

2. 기술 리서치 결과

기술 조사 과정에서는 직접 개발한 AI 리서치 도구와 공식 문서를 함께 참고하여 구조를 설계했습니다.

- **Next.js 12 조사 결과**

- Next.js 12는 Pages Router 기반 구조를 제공하며, 파일 이름이 URL 경로와 자동으로 연결되는 방식을 사용합니다. 이를 활용해 [city].js 파일만으로 /:city 형태의 동적 라우팅을 구현하고자 했습니다.
- 실시간 날씨 데이터를 항상 최신 상태로 제공하기 위해 **getServerSideProps** 기반 **SSR** 방식을 채택했습니다.

- **GraphQL**

- 과제 요구사항에 맞춰 **GraphQL** 기반 구조를 적용했습니다.
- REST 방식처럼 여러 API 경로를 나누는 대신, POST /api/graphql 단일 엔드포인트로 데이터를 처리하도록 구성합니다.
- Apollo Server를 Next.js API Route 내부에 통합해 별도의 백엔드 서버 없이 동작하도록 구현하고자 했습니다.

- **OpenWeatherMap**

- 현재 날씨는 /weather API, 5일 예보는 /forecast API를 사용합니다.
- forecast API는 UTC 기준 3시간 간격 데이터만 제공하니, 주의해야 합니다.
- weather 정보가 배열 형태로 내려오기 때문에, weather[0] 값을 기준으로 대표 날씨 상태를 추출해야 합니다.

3. 시스템 설계

1. 디렉토리 구조 및 버전 설정

코드 파일은 src/ 내부로 분리했고, 설정 파일은 루트에 유지했습니다. 코드와 설정 파일을 쉽게 구분해 확인할 수 있습니다.

Node.js 18 환경을 필요로 하는데, 개발·실행 환경에 따라 버전이 달라지면 예기치 않은 동작 차이가 생길 수 있어, .nvmrc에 Node 버전을 고정했습니다.

2. 구조

Next.js 12 기반의 단일 애플리케이션 구조로, 프론트엔드와 백엔드가 하나의 프로젝트 안에서 함께 동작하도록 설계했습니다.

GraphQL 스키마(typeDefs)와 resolver는 src/server/ 아래에 독립적으로 정의하고, 두 실행 경로가 이를 공유하는 구조로 설계했습니다. OpenWeatherMap REST API 호출과 데이터 가공은 resolver에서 전담하며, OpenWeatherMap API Key는 서버 환경 변수에서만 참조해 클라이언트에 노출되지 않도록 처리했습니다.

실행 경로는 두 가지입니다.

- SSR 경로: getServerSideProps에서 makeExecutableSchema로 스키마 객체를 생성하고, SchemaLink로 구성된 Apollo Client를 통해 GraphQL 쿼리를 실행합니다. HTTP 왕복 없이 같은 Node.js 프로세스 안에서 스키마 파싱 → validation → execution을 거치는 인프로세스 실행입니다. 쿼리 결과는 cache.extract()로 추출해 클라이언트에 전달하고, 클라이언트는 cache.restore()로 복원한 뒤 useQuery가 캐시를 읽어 /api/graphql 재요청 없이 데이터를 사용합니다.
- HTTP 엔드포인트(/api/graphql): apollo-server-micro의 ApolloServer 인스턴스를 Next.js API Route에 마운트해 POST /api/graphql 엔드포인트를 구성했습니다. 동일한 typeDefs·resolvers를 사용하며, curl·Postman 등 외부에서 직접 GraphQL 쿼리를 보낼 때 사용할 수 있습니다. 앱 런타임(SSR)에서는 이 경로를 거치지 않습니다.

4. API 설계

1. 엔드포인트

- POST /api/graphql 단일 엔드포인트

2. 쿼리 종류

- currentWeather(city) — 현재 날씨
- forecast(city) — 5일 예보 (날짜별 그룹핑)

3. 에러 처리

[SSR 레벨 — getServerSideProps]

- CITIES에 없는 도시 요청 → { notFound: true } → Next.js 기본 404 페이지
- API 호출 중 예외 발생 → { notFound: true } → Next.js 기본 404 페이지

[resolver 레벨 — GraphQL 에러]

- 허용되지 않은 도시 → CITY_NOT_FOUND
- OWM 응답 실패 → EXTERNAL_API_ERROR
- 5초 타임아웃 → REQUEST_TIMEOUT

5. UI 퍼블리싱

색상 관리의 편의성을 위해, 프로젝트에서 사용하는 주요 색상을 CSS 변수로 정의했습니다.

Weather main

Welcome to
Weather App!

Hour

Seoul

Seoul

Tokyo

Paris

London

Seoul

Seoul



Weather depth_다 접혀있을때

Weather Information for Seoul

May 23, 03:00am

Seoul, KR

292.98°C

5-day Forecast

May 23

May 24

May 25

May 26

May 27

Weather depth_한개 펼쳤을때

Weather Information for Seoul

May 23, 03:00am

Seoul, KR

292.98°C

5-day Forecast

May 23

03:00am

297.32°C / 297.32°C

06:00am

297.32°C / 297.32°C

09:00am

297.32°C / 297.32°C

12:00pm

297.32°C / 297.32°C

15:00pm

297.32°C / 297.32°C

18:00pm

297.32°C / 297.32°C

21:00pm

297.32°C / 297.32°C

May 24

May 25

May 26

May 27

4018 × 1801

Selection colors

☐ --color-white

☐ --color-darkgray

☐ --color-red

☐ --color-lightgray

☐ --color-gray

☐ --color-black

☐ --color-pink1

☐ --color-darkgray2

☐ --color-palepink

☐ --color-magenta

☐ --color-pink2

6. 백엔드 구현

백엔드 구현 결과 요약

- GraphQL 스키마는 CurrentWeather, ForecastItem, DayForecast, Forecast 타입으로 정의했으며, resolver에서 OpenWeatherMap REST API를 호출한 뒤 응답 데이터를 GraphQL 타입 구조에 맞게 가공하도록 구현했습니다.
- 5일 예보 데이터는 OpenWeatherMap API가 UTC 기준 3시간 간격 데이터를 반환하기 때문에, city.timezone 오프셋을 적용해 현지 시간 기준으로 다시 그룹핑했습니다.
- 실제 페이지 렌더링 시에는 getServerSideProps에서 makeExecutableSchema로 스키마 객체를 생성하고, SchemaLink로 구성된 Apollo Client를 통해 GET_CURRENT_WEATHER, GET_FORECAST 쿼리를 실행합니다.
이 방식은 실제 GraphQL 실행 엔진(파싱·validation·execution)을 거치므로 GraphQL 백엔드를 형식이 아닌 실질적으로 사용하는 구조입니다.
쿼리 완료 후 cache.extract()로 캐시 상태를 추출해 initialApolloState prop으로 전달하면, 클라이언트의 _app.js가 cache.restore()로 복원하고 useQuery가 캐시에서 직접 읽어 네트워크 재요청이 발생하지 않습니다.
- 또한 허용 도시 목록(CITIES)을 src/lib/constants.js로 분리해 [city].js와 resolvers.js에서 공통으로 import하도록 구성했으며, 날짜·시간 포맷 로직은 src/lib/formatTime.js로 추출했습니다.
Next.js API Route의 GraphQL 엔드포인트(POST /api/graphql)는 동일한 스키마·resolver를 공유하며, curl·Postman 등 외부 접근용으로 유지됩니다.

7. 구현 과정에서의 고민

1. 3시간 단위의 예보 데이터 표시에 대한 고민

초기 퍼블리싱을 할 때에는 피그마 디자인에 맞춰 3시간 단위 예보를 고정적으로 표시했습니다.

그러나 OpenWeather API를 연동하는 과정에서 예보는 본래 미래 데이터를 의미하므로, 오늘 날짜의 지난 시간대 예보를 포함할 경우 데이터의 성격에 맞지 않고, 사용자에게 혼란을 줄 수 있다고 판단했습니다.

시중에 나와있는 애플리케이션인 아이폰의 '날씨' 예시를 참고하여,  오늘 날짜의 예보는 **현재 시간대 이후의 데이터만 표시하도록 구현**했습니다.

2. 현재 시간의 표시에 대한 고민

OpenWeather API의 현재 날씨 응답 데이터를 가져온 결과 시간이 실제 요청 시각보다 1~9분 이전의 시각으로 제공되는 현상을 확인했습니다.

원인 분석 결과 무료 플랜에서는 기상 관측 데이터를 10분 단위로 업데이트 하고 있었고, 해당 사항을 고려하여 **현재 시각은 new Date() 기준으로 표시**하고, **날씨 정보만 최신 관측 데이터를 사용**해서 표시하도록 구현했습니다.



8. 트러블 슈팅

문제

/forecast API는 데이터를 UTC 기준 고정 시각(00, 03, 06 ... 21시)으로만 제공합니다.
이 값을 고려하지 않은 채로 필터링을 진행했고, 다음과 같은 세 가지 문제가 발생했습니다.

1. 각 도시의 현재 시각이 로컬 시각이 아닌, **UTC 시각으로 표시됨**.
2. 표시할 시각을 {3, 6, 9, 12, 15, 18, 21} 세트로 필터링했는데, 파리나 런던의 경우 로컬 시각이 UTC+2, UTC+1이기 때문에 세트 내 시간대에 하나도 해당되지 않아 **모든 예보 항목이 걸러지면서 표시되지 않는 오류가 발생함**.
3. 각 예보 항목에 포함되는 날짜+시각 문자열 필드인 dt_txt는 UTC 기준이라
서울 기준 5/23 00:00인 항목이 dt_txt상으로는 2026-05-22 15:00:00으로 표기됨.
5/22 예보 행을 펼쳤을 때 5/23 오전 시간대 예보까지 포함되는 오류가 발생함.

원인 분석

UTC로 들어온 시각을 기준으로 **필터링을 먼저 처리**한 것이 문제가 되었습니다.

해결 과정

timezone 필드(초 단위 UTC 오프셋, 서울=32400 등)를 schema와 resolver에 추가해 프론트까지 전달했습니다.

- 현재 시각을 timezone offset을 더한 뒤 getUTCHours()로 읽는 방식으로 변경해 로컬 시각으로 변환했습니다.
- 이미 3시간 간격의 데이터를 제공하고 있기 때문에, 특정 시각 세트 조건을 제거하고 현재 시각 이후 항목을 가져오도록 했습니다.
- UTC 날짜 문자열인 dt_txt 대신, dt(Unix 타임스탬프)에 timezone offset을 더해 직접 계산하는 방식으로 변경했습니다.

계산된 timezone 값은 [city].js → ForecastList → ForecastItem까지 props로 전달해 모두 동일한 기준으로 처리했습니다.

결과

각 도시들의 현지 시각이 정확하게 표시되었고, 파리·런던의 예보가 정상 출력되었고, 날짜 경계에서 발생하던 오류가 해소되었습니다.

9. 회고

준비와 설계 과정, 부족했던 시간

프로젝트로는 처음 다뤄보는 스택이었던 만큼, 초반에 전체 구조에 감을 잡는 데 시간이 걸려 준비 기간이 다소 길어졌습니다. 다만 전체 구현 기간이 짧아 기술 리서치에 충분한 시간을 들이지 못한 점이 아쉽습니다. 평소에는 설계를 깔끔하게 마친 뒤, 개발에 들어가는 편인데, 이번에는 다소 급하게 진행한 덕분에 후반부에 계속 수정을 거듭해야 했습니다. 결국 테스트 부분은 구현하지 못해 아쉬움이 남습니다.

기술적으로 배운 점

이번에 Apollo(GraphQL)를 처음 다뤄봤습니다. 아직 깊이 이해했다고 하기는 어렵지만, 클라이언트가 필요한 필드를 직접 지정해 원하는 형태로 데이터를 가져오는 방식이 흥미로웠습니다. REST에서 자원마다 엔드포인트를 두던 것과 달리, 스키마와 resolver만 정의하면 클라이언트가 필요한 조합으로 데이터를 꺼내 쓸 수 있다는 것을 배웠습니다. 앞으로 더 공부해서 적극적으로 활용하고 싶습니다.

초기 구현에서는 `getServerSideProps`에서 resolver를 직접 호출했는데, 검토하면서 이 방식이 GraphQL을 전혀 거치지 않는다는 것을 알게되었습니다. Apollo를 설치하고 Provider까지 감싸뒀지만, 데이터가 그 경로를 지나가지 않고 있었습니다. 이를 수정하면서 `SchemaLink`를 알게 됐습니다. 서버 안에서 바로 GraphQL을 실행하고, 그 결과를 캐시에 담아 클라이언트로 넘기는 방식입니다. 클라이언트는 이 캐시를 그대로 복원해 쓰기 때문에, 화면의 `useQuery`가 데이터를 다시 요청하지 않습니다. 실제로 네트워크 요청을 확인해보니 `/api/graphql` 호출이 0건이었습니다. 라이브러리를 설치하고 Provider로 감싸기만 하면 쓰는 거라고 생각했는데, 데이터가 정말 그 경로를 지나가는지 확인하는 게 더 중요하다는 걸 배웠습니다.

저는 기술을 통해 사용자에게 더 좋은 경험과 가치를 전달할 수 있는 개발자를 지향합니다.

백지은
jieun13.b@gmail.com

2026-05